JavaScript Day 5 – Modern JavaScript

Detailed Theory Notes: Template Literals • Optional Chaining • Ternary Operator • try...catch • Local Storage • JSON • Revision

Note: Day 5 practical task is intentionally not included.

1. Template Literals

Template literals make strings easier to write when they contain variables or expressions. They use backticks (`) instead of single or double quotes.

Normal concatenation

```
let name = "Namratha";
console.log("Hello " + name);
```

Template literal

```
let name = "Namratha";
console.log(`Hello ${name}`);
```

Expression inside a template

```
let a = 10;
let b = 20;
console.log(`Total = ${a + b}`);
```

Remember: Template literal = backticks + ${expression}.

2. Optional Chaining (?.)

Optional chaining safely accesses a property or method when an earlier value may be null or undefined. If the chain cannot continue, it returns undefined instead of throwing an error.

Possible problem

```
let employee = { name: "Rohit" };
console.log(employee.address.city);
```

Safer access

```
let employee = { name: "Rohit" };
console.log(employee.address?.city);
```

Nested example

```
let user = { profile: { name: "Anu" } };
console.log(user.profile?.name);
console.log(user.contact?.phone);
```

Remember: ?. means “continue only if the value exists.”

3. Ternary Operator

The ternary operator is a short form of a simple if...else decision. It is useful when choosing between two values.

Normal if...else

```
let age = 20;
if (age >= 18) {
result = "Adult";
} else {
result = "Minor";
}
```

Ternary

```
let age = 20;
let result = age >= 18 ? "Adult" : "Minor";
```

Another example

```
let marks = 75;
let result = marks >= 40 ? "Pass" : "Fail";
```

Pattern: condition ? valueIfTrue : valueIfFalse. Avoid deeply nested ternaries because they are harder to read.

4. Error Handling – try...catch

try...catch lets you handle certain runtime errors. Put code that may throw an error inside try; if an error is thrown, catch runs.

Basic structure

```
try {
// code that may cause an error
} catch (error) {
// handle the error
}
```

Example

```
try {
let result = JSON.parse("wrong data");
console.log(result);
} catch (error) {
console.log("Invalid JSON");
}
```

Optional finally

```
try {
console.log("Start");
} catch (error) {
console.log("Error");
} finally {
console.log("This runs afterward");
}
```

Remember: try = attempt risky code; catch = handle the thrown error; finally = optional code that runs afterward.

5. Local Storage Basics

Local Storage lets a web page save small amounts of data in the browser as key-value pairs. Values normally remain after refresh until removed.

Save

```
localStorage.setItem("name", "Namratha");
```

Read

```
let name = localStorage.getItem("name");
console.log(name);
```

Remove one item

```
localStorage.removeItem("name");
```

Clear all local storage

```
localStorage.clear();
```

Important: Local Storage stores values as strings. Use JSON conversion for arrays and objects.

6. Introduction to JSON

JSON stands for JavaScript Object Notation. It is a text format used to represent structured data. It looks similar to JavaScript objects, but JSON itself is text.

JavaScript object

```
let employee = {
name: "Rohit",
age: 25
};
```

Object → JSON string

```
let data = JSON.stringify(employee);
console.log(data);
```

JSON string → object

```
let employeeAgain = JSON.parse(data);
console.log(employeeAgain.name);
```

Remember: JSON.stringify() = JavaScript data → JSON string. JSON.parse() = JSON string → JavaScript data.

7. Local Storage + JSON Together

Because Local Storage stores strings, JSON is commonly used to save arrays or objects.

Save an array

```
let tasks = ["Study", "Exercise", "Read"];
localStorage.setItem("tasks", JSON.stringify(tasks));
```

Read the array back

```
let savedTasks = JSON.parse(localStorage.getItem("tasks"));
console.log(savedTasks);
```

Flow: array/object → JSON.stringify() → string → Local Storage. Reading: Local Storage → string → JSON.parse() → array/object.

8. Revision of JavaScript Fundamentals

Concept	Key idea	Example
Variables	Store values	let age = 22;
Data types	String, Number, Boolean, etc.	let name = "Sam";
Operators	Calculate or compare	a + b, a > b
if / else	Make decisions	if (age >= 18) {...}
switch	Choose among cases	switch(day) {...}
Loops	Repeat code	for (...) {...}
Functions	Reusable code	function add(a,b) {...}
Arrays	Store multiple values	let items = [1,2,3];
Objects	Store related properties	let user = {name:"Sam"}
DOM	Find/change HTML	document.getElementById(...)
Events	React to actions	addEventListener(...)

Quick Revision Cheat Sheet

Feature	Remember
Template literals	`backticks` + ${expression}
Optional chaining	?. safely accesses a possibly missing value
Ternary	condition ? trueValue : falseValue
try...catch	TRY risky code; CATCH the error
Local Storage	setItem / getItem / removeItem / clear
JSON	stringify = data → JSON string; parse = JSON string → data